UNIT 3

Exercise 8: Context-aware chatbot using LangChain, retrieval, and an LLM

Aim

To generate numerical embeddings for text and identify semantically similar sentences using embedding similarity.

Algorithm

- Start the program.
- Load a pretrained sentence-embedding model.
- Define a collection of text documents.
- Convert the documents into numerical vectors.
- Convert the user's query into an embedding.
- Calculate similarity scores between the query and documents.
- Display the most similar document.
- Stop.

Python Code and Output

```
    # Use the existing embedding_model and chunks to retrieve similarity search r
    q_vector = embedding_model.encode([question], normalize_embeddings=True)[0]
    scores = chunk_vectors @ q_vector
    top_indices = np.argsort(scores)[::-1][:3]
    context = "\n".join(chunks[i] for i in top_indices)

    history_text = "\n".join(
        f"User: {q}\nAssistant: {a}"
        for q, a in history[-3:]
    )

    combined_context = (
        f"Retrieved documents:\n{context}\n\n"
        f"Recent conversation:\n{history_text}"
    )

    answer = generate_answer(question, combined_context)

    print("Chatbot:", answer)
    history.append((question, answer))
```

```
You:  What is machine learning?
Chatbot: Machine learning allows comp learning allows computers to learn patterns from data

You: Which field uses neural networks?
Chatbot: Deep learning

You: exit
Chatbot: Goodbye!
```

Result

Text embeddings were generated successfully, and semantic similarity search identified the most relevant document.